

# Concurrent Data Structure (AST KD-Tree)

Alex Lee

# Problem Definition

## What is a KD-Tree?

- A K-dimensional tree (KD-tree) is a spatial data structure used to organize points in multi-dimensional space.
- It supports efficient range queries, nearest neighbor (k-NN) search, and spatial partitioning.
- Commonly used in:
  - Robotics (collision checking, SLAM)
  - Computer Vision (image retrieval, feature matching)
  - Real-time Systems (sensor fusion, planning)

## Limitations of Traditional KD-Trees

- Perform poorly in high dimensions (curse of dimensionality)
- Sequential nature limits scalability on multi-core systems
- Struggle with dynamic or frequently updated datasets

## The Core Problem

How can we design a KD-tree structure that supports high-dimensional, large-scale, and dynamic data efficiently—while leveraging hardware parallelism for fast queries?

# Survey

## Acceleration based Ideas

### Distributed Parallel KD-Tree

- Used in astrophysics (N-body simulations)
- Parallel Sorting: Global sort
- Pros: Scales to 512+ processors
- Limitations: Only 3D data, preprocessing for sorting

### BDL-Tree (Batch-Dynamic Log KD-tree)

- Combines buffer trees + static subtrees
- Supports batched: insert, delete, k-NN
- Pros: Great dynamic update support, data-parallel
- Cons: Multiple static trees, High query cost in high-dimensions

### Skeleton-Based Batch Insertion

- Updates shallow “skeleton” then rebuilds affected parts
- Pros: Good for insert-heavy, dynamic workloads
- Cons: No support for delete or k-NN; not general-purpose

## Accuracy based Ideas

### Perturbed Querying

- Add Gaussian noise → repeat KD-tree query
- Pros: Boosts NN hit rate with no tree change
- Cons: Slower runtime due to multiple queries

### Hierarchical Indexing

- Use Hashing + KD-tree with position-based splits
- Pros: Coarse-to-fine filtering for SIFT features
- Cons: Tailored to image data; poor generalization

### RKD-Tree

- KD-tree with bounding spheres + triangle inequality
- Pros: Fewer false-positive subtree visits
- Cons: High memory; designed for static datasets only

# Proposed Idea

## Goal:

Design a high-performance KD-tree supporting:

- Faster k-NN and range queries
- Efficient dynamic updates
- Robustness in high-dimensional settings

## Motivation:

- Traditional KD-trees degrade with high dimensions
- Most existing KD-trees are static
- Need scalability, adaptability, and speed for real-time systems

## Key Ideas:

1. Subspace Projection for dimensionality reduction (Adaptive Subspace Tree)
2. Parallel Construction & Search using OpenMP (parallel for, recursive tree building)
3. Support for Insertions/Deletions (dynamic updates)

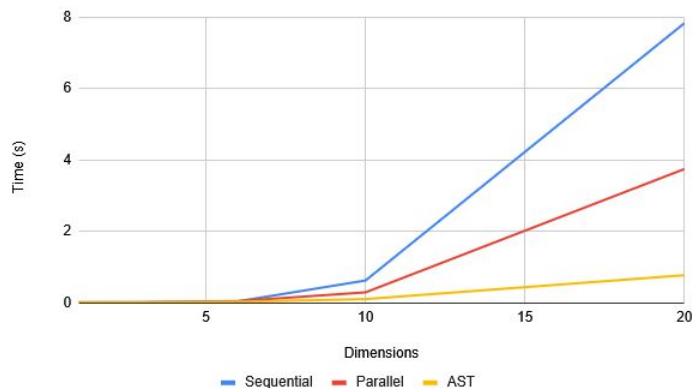
# Results and Analysis

## Test 1 (Batched k-NN Benchmark)

Time vs Number of k-NN queries with Constant Tree Size (100,000), Dimensions (10) and Threads (4)

|            | 1        | 50       | 100     | 500      | 5000     |
|------------|----------|----------|---------|----------|----------|
| Sequential | 0.002224 | 0.055915 | .116927 | 1.286059 | 6.044492 |
| Parallel   | 0.006334 | 0.046664 | .068487 | 0.513622 | 2.588069 |
| AST-Flat   | 0.005567 | 0.014945 | .024759 | 0.120447 | 0.579995 |

Time vs Dimensions with Constant Tree Size (100,000), Queries (500) and Threads (4)

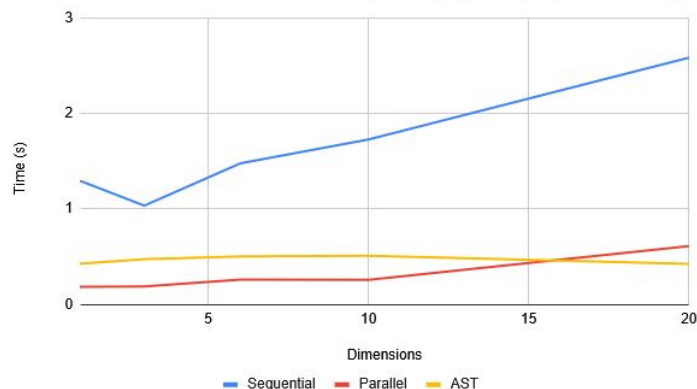


## Test 2 (Batched Range Search Benchmark)

Time vs Number of Range queries with Constant Tree Size (100,000), Dimensions (10) and Threads (4)

|            | 1        | 50       | 100      | 500      | 5000      |
|------------|----------|----------|----------|----------|-----------|
| Sequential | 0.002904 | 0.202122 | 0.321137 | 2.248381 | 19.491016 |
| Parallel   | 0.015750 | 0.197207 | 0.213487 | 0.916284 | 6.910382  |
| AST-Flat   | 0.012878 | 0.205291 | 0.427842 | 1.612269 | 9.808697  |

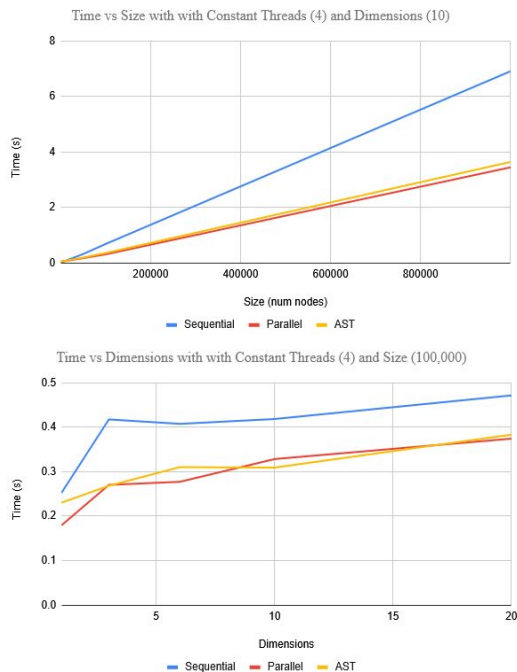
Time vs Dimensions with Constant Tree Size (100,000), Queries (500) and Threads (4)



For high dimensionality and high query count, large decrease in time taken; slightly higher time at low counts from overhead; batched range search has a higher time but is more dependent on data while kNN is more predictable

# Results and Analysis (cont...)

## Test 3 (Tree Construction)



## Test 4 (Other metrics)

### Time vs Number of Points to Insert/Delete with Constant Dimensions (10)

|          | 10      | 500     | 50,000  | 100,000 | 1,000,000 |
|----------|---------|---------|---------|---------|-----------|
| Normal   | .000028 | .000807 | .307982 | .725990 | 8.968653  |
| AST-Flat | .000025 | .001010 | .294455 | .951928 | 8.504750  |

### k-NN test - Average distance to 3-NN vs dimensions (100,000 points)

|              | 1 | 3      | 6       | 10      | 20      |
|--------------|---|--------|---------|---------|---------|
| Trad KD-Tree | 0 | 1.5051 | 12.0575 | 32.1922 | 81.4069 |
| AST-Flat     | 0 | 1.5060 | 12.0609 | 32.2630 | 81.6995 |

### Range test - Num points detected vs dimensions (100,000 points)

|              | 1     | 3     | 6     | 10  | 20 |
|--------------|-------|-------|-------|-----|----|
| Trad KD-Tree | 36184 | 47208 | 23663 | 364 | 0  |
| AST-Flat     | 36184 | 47208 | 23663 | 364 | 0  |

Tree construction is relative to traditional methods even for large sizes, without decreasing in accuracy

# Conclusions

- AST KD-tree offers the best scalability for large-scale batched queries at high dimensions (k-NN and range), significantly outperforming both sequential and traditional parallel implementations as query count, point size, or thread count increases without a significant drop in accuracy at the cost of slight increase in performance at lower values of those variables.
- Parallelization provides consistent speedups for batched operations and tree construction, but benefits plateau beyond a certain thread count due to memory bandwidth and task overhead — especially for shallow or small datasets.
- AST remains competitive even with dynamic operations (insert/delete), scaling well with large datasets despite not being inherently designed for such use, making it a viable hybrid structure for mixed workloads.